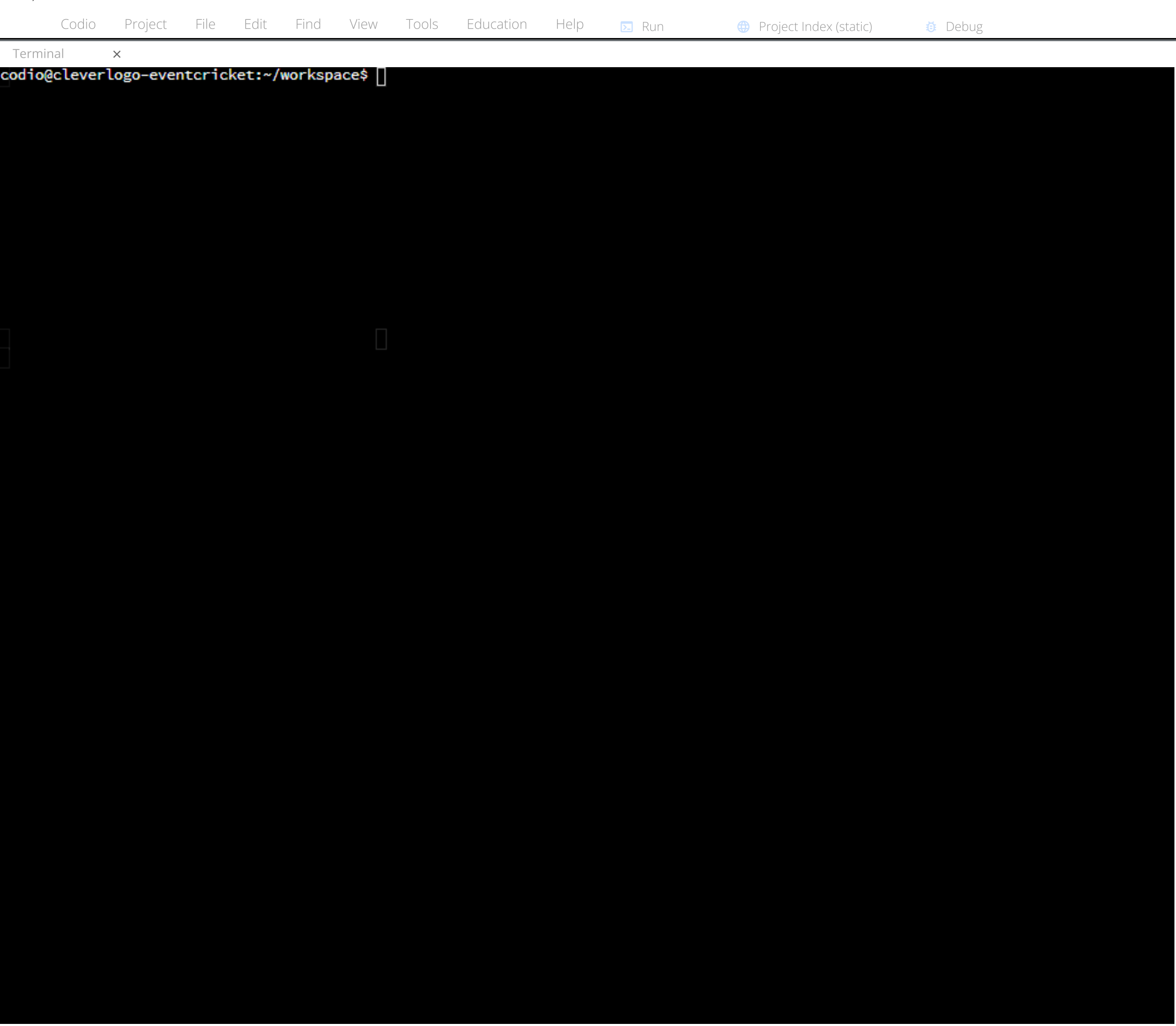




Guide

Collapse

The Lock File

Dependencies of Dependencies

Start by changing into the `learning-poetry` directory.

```
cd learning-poetry
```

If you look at the dependencies in the `pyproject.toml` file, you would think this Poetry project has only four dependencies. However, the `fire` package has dependencies of its own, but you cannot see them.

```
[tool.poetry.dependencies]
python = "^3.6.2"
shapes = {git = "git@github.com:codio-content/shapes-package.git"}
idna = "^3.3"
fire = "^0.4.0"
```

Now compare the above information when using the `show --tree` command to view a hierarchical structure of the installed packages.

```
poetry show --tree
```

You should see `six` and `termcolor` listed as dependencies of the `fire` package. Poetry stores the complete list of dependencies (and their dependencies) in the `poetry.lock` file. Click the link below to see what this looks like.

Open `poetry.lock`

One difference between dependency versions listed in `pyproject.toml` and those in `poetry.lock` is the absence of the `^` character. The lock file specifies the exact version to be installed. In the example below, Poetry will install only version 1.4.0 of `atomicwrites`.

```
[[package]]
name = "atomicwrites"
version = "1.4.1"
description = "Atomic file writes."
category = "dev"
optional = false
python_versions = ">=2.7, !=3.0.*, !=3.1.*, !=3.2.*, !=3.3.*"
```

Why is there a difference? Imagine if `package_a` lists `atomicwrites = ">1.3"` as a dependency and `package_b` lists `atomicwrites = ">1.4"` as its dependency. Both packages need `atomicwrites`, but they have different requirements as to the version. Poetry resolves dependencies before installing them. That means Poetry will find a version of `atomicwrites` that satisfies both `package_a` and `package_b`. This version is then saved to `poetry.lock`.

The lock file is an important part of your Poetry project. If you are sharing your work with others, be sure to include `poetry.lock` so they can install the same package versions as you.

► What happens if there is no lock file?

Poetry and a Requirements File

Assume that you use Poetry for all your Python projects. You want to share your latest one with a friend. However, they use `pip` for their package management. The `poetry.lock` file is useless to them since they expect a `requirements.txt` file to contain the dependencies. Thankfully, Poetry can produce a requirements file.

Open terminal

```
poetry export --output requirements.txt
```

Use the `cat` commands to print the text of the requirements file to the terminal. Even though Poetry has its own way of doing things, that does not mean your work is isolated from the larger Python community.

```
cat requirements.txt
```

Interacting with the larger Python community means that Poetry needs to be able to install from the `requirements.txt` file as well. The project `use-requirements` has already been created using Poetry. Change into this directory and display the hierarchical structure.

```
cd ../use-requirements && tree
```

You should see a `requirements.txt` file in the directory. We are going to use this to do the initial installation of dependencies. The command below sends the contents of the requirements file to the `add` command for Poetry.

```
poetry add $(cat requirements.txt)
```

Now use Poetry's `show` command to list the dependences installed for the project.

```
poetry show --tree
```

The `poetry.lock` file stores the exhaustive list of dependencies in a Poetry project. Every Poetry project should have one of these files. In addition, Poetry can create or install from a `requirements.txt` file if need be.

Reading Question

Fill in the blanks with either True or False.

- False - `pyproject.toml` is a complete list of dependencies
- True - You can install dependencies without a `poetry.lock` file
- True - Poetry can install from a `requirements.txt` file
- TrueFalse - You should include the `poetry.lock` file when sharing a Poetry project

The correct answers are:

- False - `pyproject.toml` is a complete list of dependencies
- True - You can install dependencies without a `poetry.lock` file
- True - Poetry can install from a `requirements.txt` file
- False - You should include the `poetry.lock` file when sharing a Poetry project

The `pyproject.toml` file only contains the names of dependencies for the project. It does not contain the names of dependencies for the dependencies. Those are stored in the `poetry.lock` file.

Poetry can install dependencies if there is no `poetry.lock` file. Some issues could arise since Poetry will install the latest version of each dependency as opposed to the very specific version specified in the lock file.

Poetry can use a `requirements.txt` file to install dependencies.

You should always include the lock file when sharing your Poetry project. This ensures that other users use the same dependencies as you and avoid an unexpected issues.

Check it!

Next